

ASCL HANDS-ON TRAINING · DAY 4 OF 4

ROS2

ROS 2 & LIMO Cobot Training

From Robot Concepts to Real-World Action

LAB	Automated Systems & Computing Lab (ASCL)
LAB LEADER	Prof. Ahmad Taher Azar

Dr Saim Ahmed
Trainer 1 · Day 1 & Day 3

Eng. Zeeshan Haider
Trainer 2 · Day 2 & Day 4



ASCL
Automated Systems & Computing Lab

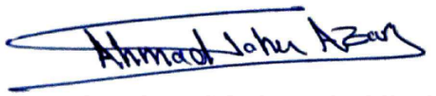
جامعة الامير سلطان
PRINCE SULTAN
UNIVERSITY



ACKNOWLEDGMENTS

This Training is supported by a research grant funded by the Research, Development, and Innovation Authority (RDIA), Kingdom of Saudi Arabia, with grant number 13382-psu-2023- PSNU-R-3-1-EI.

Prof. Ahmad Taher Azar



Full Professor, Leader of Automated Systems & Computing Lab (ASCL)

TRAINING AGENDA · 4 DAYS

Table of Contents

TRAINING 1 · DAYS 1–2 · FUNDAMENTALS

01 **Day 1 · What is ROS & ROS2?**

- Sense → Think → Act & the group-chat model
- Core concepts: nodes, topics, messages, services, actions
- turtlesim hands-on lab

02 **Day 2 · ROS2 Hands-On: CLI & First Nodes**

- The `ros2` CLI: node · topic · service · param
- Workspaces & packages
- First publisher & subscriber in Python

TRAINING 2 · DAYS 3–4 · ON THE ROBOT

03 **Day 3 · Meet the LIMO Cobot: Hardware & Bring-Up**

- Hardware, motion modes & sensors
- Connect & bring-up: NoMachine + chassis driver
- Drive & sense

04 **Day 4 · Applications: SLAM, Navigation & the Arm**

- SLAM mapping with Cartographer
- Navigation with Nav2
- myCobot 280 arm control

ROADMAP

Today's plan

01
SLAM Mapping
The robot draws its own map. 15 min

02
Navigation
Click a goal — it drives itself. 15 min

03
The Cobot Arm
Control the arm, see trajectory following.
15 min

Three demos, one theme: sensing → understanding → acting.

BY THE END OF TODAY

You will be able to...

- Explain **SLAM** in one sentence and build a map with Cartographer
- Save a map and use it to **navigate** with Nav2
- Set an initial pose and send a **navigation goal** in RViz2
- Control the **myCobot arm**

01

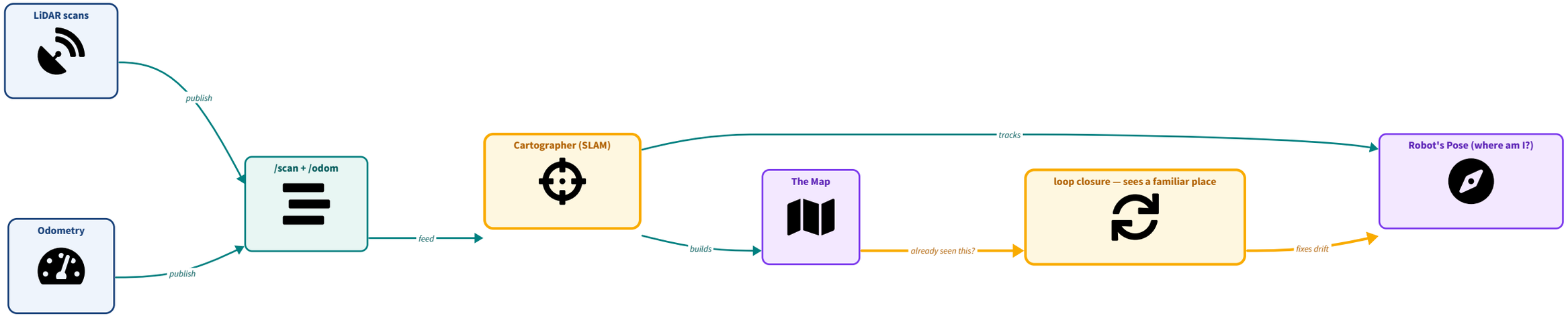
SLAM Mapping

The robot draws its own map — and knows where it is on it.

THE PUZZLE

SLAM

SLAM (Simultaneous Localization and Mapping) in one sentence:
the robot draws the map **while** figuring out where it is on that map — both at the same time.



LiDAR + odometry → builds the map AND tracks the pose · the amber loop is

THE 4-STEP WORKFLOW

Build & save a map

1. **Start sensors + base** — `ros2 launch limo_bringup limo_start.launch.py`
2. **Start Cartographer + RViz** — `ros2 launch limo_bringup cartographer.launch.py`
3. **Drive the room** — `ros2 run teleop_twist_keyboard teleop_twist_keyboard`
4. **Save the map** — `ros2 run nav2_map_server map_saver_cli -f map`

STEPS 1–2

Start sensors & Cartographer

Terminal 1 — sensors + base

leave running

```
ros2 launch limo_bringup \
  limo_start.launch.py
```

Terminal 2 — mapping + RViz

leave running

```
ros2 launch limo_bringup \
  cartographer.launch.py
```

RViz opens with a blank map view — **the robot hasn't seen the room yet.**

> Distro note: Foxy = `cartographer.launch.py`; on Humble try `limo_cartographer.launch.py`.

STEP 3

Drive the room

Terminal 3 — teleop

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

1. **Follow the walls** first (frames the room)
2. **Criss-cross the middle**
3. **Return to where you started** — triggers loop closure, tightens the map

Watch RViz: black = walls, white = free space. You're watching the robot's brain draw the room.

> ⚠️ **Slow is smooth, smooth is fast.** Mapping at speed produces a messy map.

STEP 4

Save the map

New terminal

```
ros2 run nav2_map_server map_saver_cli -f map
```

`map.pgm`

the picture (black/white grid)

`map.yaml`

the metadata (resolution, origin)

These two files are **the robot's memory of the room** — the input for navigation next.

New terminal

```
colcon build --packages-select limo_bringup
```

```
source install/setup.bash
```

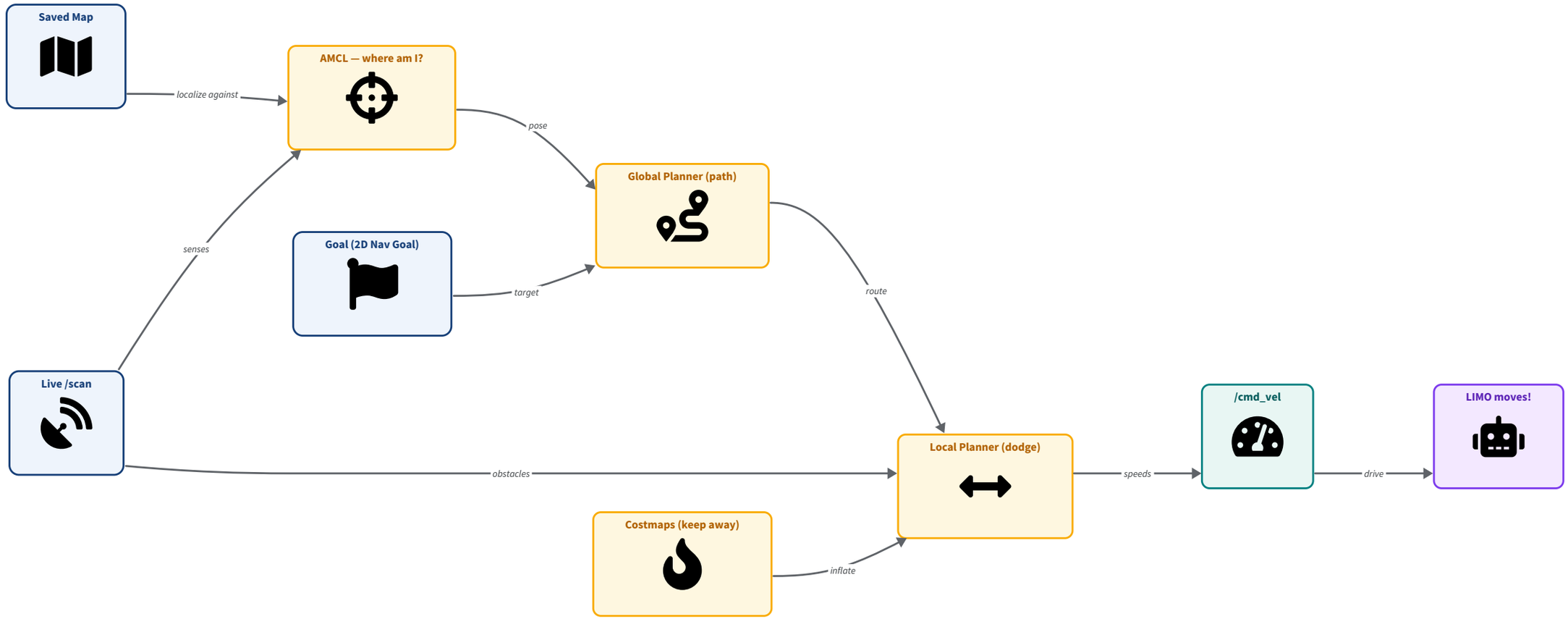
02

Navigation

Click a point on the map — the robot drives itself there.

FROM MAP TO MOTION

The Nav2 pipeline



AMCL → global planner → local planner → costmaps → `/cmd_vel`

In plain words: **localize → plan the route → dodge as you go → drive.**

THE PIECES

What each Nav2 part does

Piece	One-liner
AMCL (Adaptive Monte Carlo Localization)	"Where am I?" — matches LiDAR to the saved map
Global planner	The GPS route to the goal
Local planner	Moment-to-moment dodging
Costmap	"Keep away" zones inflated around obstacles
Planned path	The robot's plan, drawn for you to see

LAUNCH

Start navigation

Terminal 1 — sensors + base

same as mapping

```
ros2 launch limo_bringup \
  limo_start.launch.py
```

Terminal 2 — Nav2

RViz opens with saved map

```
ros2 launch limo_bringup limo_nav2.launch.py  \
map:=/home/agilex/limo_ros2_ws/src/limo_ros2/limo_bringup/maps/map.yaml
```

> Ackermann mode? Use `limo_nav2_ackmann.launch.py` (Foxy name) instead.

STEP 1

Tell the robot where it starts

1. In RViz2, click **2D Pose Estimate** (toolbar)
2. Click where the robot **actually is**, drag to point the arrow in its **facing direction**
3. Release — watch the laser points (red dots) snap toward the map walls

> **Roughly is fine:** AMCL uses a cloud of particles that converge on the correct pose. You give a guess; it refines it. If the laser doesn't line up, set the pose again.

STEP 2

Send a goal & watch the magic

1. Click **2D Nav Goal** (toolbar)
2. Click a clear spot, drag to set final **facing direction**
3. Release — a **planned path** appears, and the robot **drives itself there**

The colored line is the global plan. Watch the local planner steer around anything unexpected. **You did nothing but point.** 🎉

Everyone gets a turn — send goals to different corners, even behind a chair.

BONUS

Multi-point navigation

Queue several goals to form a **delivery route**:

1. Enter multi-point mode (Nav2 Goal button in the RViz panel)
2. Click several points on the map
3. Start — the robot visits them in order

> This is literally how warehouse and delivery robots plan routes — queue a path, execute, repeat.

03

The Cobot Arm

The robot grows a hand — and the whole course ties together.

THE HAND

myCobot 280 M5

A 6-joint arm from Elephant Robotics, small enough for a desk.

Spec	Value	Plain meaning
6 DOF	6 joints	Moves like a human arm (shoulder → wrist)
Payload	250 g	Can lift a small object
Reach	280 mm	Works within a desk-sized circle
End effectors	Gripper, suction pump	The "hand" — grab or suck

⚠️ The arm moves fast and can pinch — keep hands clear; never grab it while it's moving.

THREE WAYS, EASIEST FIRST

Control the arm

1 · Calibration

One calibrate each joint.

2 · Trajectory by Robot

Move in 3D space (x, y, z, rotations).

3 · Trajectory using code

Give coordinates to follow

PYTHON PROGRAM

Direct joint control

Terminal

```
python robot_dance.py
```

The desired trajectory given to robotic arm; the joints move accordingly.



Final Check

Five minutes. Pass ≥ 70% — answers revealed on each slide.

Q1 · What does SLAM stand for, and what does it solve?

A) Simultaneous Laser And Motion — moves while scanning

B) Simultaneous Localization and Mapping — maps while tracking where it is

C) Slow Localization And Mapping — maps at low speed

D) Single Localization And Motion — one sensor only

▸ **Show answer**

Q2 · What is "loop closure" in Cartographer?

A) The robot stops at the map edge

B) Recognizing a place it visited before and correcting drift

C) Closing the robot's gripper

D) Shutting down mapping

▸ **Show answer**

Q3 · Which TWO files does `map_saver_cli -f map` produce?

A) `map.slam` and `map.pose`

B) `map.pgm` and `map.yaml`

C) `map.pdf` and `map.png`

D) `map.txt` and `map.csv`

▸ **Show answer**

Q4 · Which Nav2 component answers "where am I on the map?"

A) Global planner

B) Local planner

C) AMCL

D) Costmap

▸ **Show answer**

Q6 · Order a full mapping session:

1. `map_saver_cli -f map`
2. `cartographer.launch.py`
3. `limo_start.launch.py`
4. `teleop_twist_keyboard`

▸ Show answer

Q7 · After launching Nav2, which RViz action comes FIRST?

A) 2D Nav Goal

B) 2D Pose Estimate

C) Add LaserScan display

D) Save the map

▸ **Show answer**

CAPSTONE IDEA

Design a delivery robot

Bring an object from point A to point B across a room — name the steps and the ROS2 tool for each.

1. **Build a map** — Cartographer (`cartographer.launch.py`)
2. **Save the map** — `map_saver_cli -f map`
3. **Launch Nav2** — `limo_nav2.launch.py`
4. **Set initial pose** — 2D Pose Estimate → AMCL
5. **Navigate to A** — 2D Nav Goal → planners → `/cmd_vel`
6. **Pick up** — MoveIt2 + `mycobot_280` → gripper
7. **Navigate to B** — another 2D Nav Goal
8. **Place** — reverse arm motion → release gripper

🎉 COURSE COMPLETE — ALL 4 DAYS

You did it.

From **zero ROS knowledge** to a robot that **maps a room, navigates itself, and picks up objects** — using the same topics, nodes, and messages you learned on Day 1.

Next: build the capstone

Go deeper with The Construct

100+ research papers use LIMO